Motor de anotaciones al margen

Guía de uso para un PDF cualquiera (papers, capítulos de libro)

Escrita para una IA que retome este trabajo · octubre de 2026

Qué hace. Toma un PDF y produce otro PDF donde cada página original aparece **intacta** (incrustada como gráfico vectorial, sin rasterizar) con un **margen agregado** a la derecha (y opcionalmente a la izquierda). En ese margen van notas en recuadros de colores. Cada nota se ancla a un pasaje del original con una **barra de color** y una **línea conectora**, y opcionalmente **enmarca** una ecuación o figura. Las notas se acomodan solas en el margen para no superponerse. Se creó para anotar el paper de Serrin (1959) y dos capítulos de Joseph, remitiendo a un apunte de estudio, pero sirve para cualquier PDF.

A quién va dirigida. A una IA (o una persona) que tiene que anotar un PDF nuevo. Supone que sabes leer código Typst y ejecutar Python.

1. Contenido de la carpeta

`motor.typ`	El motor: funciones `n`, `hoja`, `anotador`, `portada`, `leyenda` y el diccionario `tipos-base`. **No hace falta editarlo** para un documento nuevo.
`herramientas.py`	Script de apoyo (PyMuPDF): tamaño de página, borde del texto sugerido, coordenadas de cada línea, recortes, cuadrícula y revisión del resultado.
`ejemplo/original.typ`, `.pdf`	Un «paper» de juguete de dos páginas (la segunda a dos columnas).
`ejemplo/anotado.typ`, `.pdf`	Ejemplo completo: tipos propios, color cambiado, ambos márgenes y portada. **Cópialo como punto de partida.**
`GUIA.typ`, `GUIA.pdf`	Este documento.

2. Requisitos

- **Typst ≥ 0.14**, porque el motor incrusta páginas con `image("x.pdf", page: k)`, y los PDF como imagen llegaron en 0.14. Probado con Typst 0.15.1.
- En el computador donde se creó, el comando `typst` **no está en el PATH**: se usa el que trae la extensión Tinymist de VS Code. Busca `tinymist.exe` dentro de `~/.vscode/extensions/myriad-dreamin.tinymist-*/out/` y usa su subcomando `compile`, que tiene las mismas opciones que `typst compile`.
- **Python con PyMuPDF** (`import fitz`) para `herramientas.py`.
- Fuente de las notas: `("Segoe UI", "Arial", "New Computer Modern")`, la primera que exista. Se cambia con el parámetro `fuente:`.

> **Siempre compila con `--root`** apuntando a una carpeta que contenga **a la vez** el `.typ` y el PDF original, y escribe la ruta del PDF como absoluta desde esa raíz (`"/carpeta/paper.pdf"`). Sin `--root`, Tinymist falla con «entry path must be a valid virtual path». Typst no puede leer archivos fuera de la raíz.

```
tinymist compile --root <carpeta-raíz> <raíz>/notas/paper.typ "<salida>.pdf"
```

3. Flujo de trabajo paso a paso

Paso 1. Inspeccionar el PDF.

```
python -I herramientas.py info paper.pdf
```

Entrega `W` y `H` (tamaño de página en pt), cuántas palabras tiene la capa de texto de cada página y el **xr sugerido** (borde derecho del texto + 4 pt, donde irán las barras) y el **xl sugerido** (borde izquierdo). Si una página tiene 0 palabras es un escaneo sin OCR (ver la §8). Si avisa que `cropbox` ≠ `mediabox`, haz la prueba de alineación de la §8.

Paso 2. Obtener coordenadas verticales de los pasajes.

```
python -I herramientas.py lineas paper.pdf 1-4
```

Cada línea del texto sale como `y0-y1  x0-x1 | texto`. Las coordenadas están en pt, con origen arriba a la izquierda: **son exactamente las que usa el motor**. Para anclar una nota a un párrafo, toma el `y0` de su primera línea y el `y1` de la última. Para enmarcar una ecuación usa su rectángulo (x0, y0, x1, y1).

Paso 3. Mirar lo que el OCR destroza. En los escaneos, las ecuaciones salen como basura (`(t4)` en vez de `(14)`, letras griegas perdidas). Su **posición** suele ser correcta, pero para leerlas haz un recorte:

```
python -I herramientas.py recorte paper.pdf 3 70 100 440 200 recorte.png
python -I herramientas.py cuadricula paper.pdf 3 pagina3.png
```

`cuadricula` dibuja reglas cada 25 pt con su valor de `y`: sirve para leer coordenadas a ojo cuando no hay capa de texto.

Paso 4. Escribir el archivo de notas (ver la §4 y el ejemplo). Recomendado: un `.typ` por documento, con un bloque `#pag(k, (...))` por página, en orden.

Paso 5. Compilar y revisar.

```
python -I herramientas.py revisar anotado.pdf carpeta_revision 60
```

Renderiza el resultado de a pares de páginas en PNG y lista las páginas con **DESBORDE** (notas que no caben en el alto de la página). **Mira todas las imágenes**: comprueba que cada barra esté junto a su pasaje, que los recuadros enmarquen la ecuación correcta y que ninguna conexión cruce figuras.

Paso 6. Iterar. Corrige anclas, acorta notas, cambia `xr` en páginas con figuras y vuelve a compilar.

4. Referencia de la API

4.1. `n(y0, y1, tipo, cuerpo, r: none, lado: "der")` — una nota

- `y0`, `y1`: rango vertical del pasaje en el original (pt). La barra cubre ese rango y la nota se intenta alinear con `y0`.
- `tipo`: string, clave del diccionario de tipos (`"apunte"`, `"cuidado"`, …). Si no existe, la compilación se detiene con un mensaje que lista los tipos disponibles.
- `cuerpo`: contenido Typst. Admite `$matemática$`, `*negrita*` y `_cursiva_`.
- `r`: rectángulo `(x0, y0, x1, y1)` que se enmarca con el color del tipo (el motor agrega 2 pt de holgura). Úsalo para ecuaciones o figuras concretas, no para párrafos.
- `lado`: `"der"` (por defecto) o `"izq"`. Las notas izquierdas exigen `ML > 0` y `xl`.

4.2. `hoja(src, num, notas, W:, H:, xr:, M: 230, ML: 0, xl: none, tipos:, fuente:, tam: 7.3pt, fondo:, gap: 4pt)` — una página

- `src`: ruta al PDF original, absoluta desde `--root`. `num`: página (la primera es 1).
- `W`, `H`: tamaño de la página original en pt, **obligatorios** (lo da `info`).
- `xr`: x de las barras del margen derecho. Debe quedar a la derecha del texto y dentro del blanco del original.
- `M`: ancho del margen derecho agregado (230 pt va bien; con menos de 180 las notas quedan muy angostas).
- `ML`, `xl`: ancho del margen izquierdo (0 = sin él) y borde izquierdo del texto.
- `tipos`: diccionario de tipos (ver la §5). `tam`: tamaño de letra de las notas. `gap`: separación vertical mínima entre notas.

Las notas se numeran por su `y0` (de arriba abajo) y luego se separan por lado. Cuando dos anclas se superponen, sus barras se corren a «carriles» paralelos. Si no caben, el motor escribe `DESBORDE` en rojo arriba del margen; `revisar` lo detecta.

4.3. `anotador(src, ..fijos)` — atajo

Devuelve una función `pag(num, notas, ..cambios)` con los parámetros fijados una vez; cualquiera se puede sobrescribir por página:

```
#let pag = anotador("/papers/x.pdf", W: 504, H: 705.28, xr: 433, tipos: mis-tipos)
#pag(3, (n(84, 131, "apunte")[...], n(205, 240, "idea", r: (140, 113, 428, 131))[...]))
#pag(7, (...), xr: 372)           // esta página tiene el texto más angosto
```

4.4. `portada(titulo, subtitulo, cuerpo, W:, H:, M:, ML: 0, tipos:, encabezado:, derecha: none)`

Una página del mismo tamaño que las anotadas, con título, texto libre, la **leyenda de colores** generada desde `tipos` y una explicación de cómo leer. `derecha` es contenido opcional para una segunda columna (por ejemplo, una tabla con las etiquetas del material de apoyo). `leyenda(tipos: ...)` también se puede usar sola.

5. Crear nuevos tipos de notas (colores)

Un tipo es una entrada del diccionario `tipos`, con tres campos:

```
nombre: (color: rgb("#0f766e"), rotulo: "DEFINICIÓN", desc: [Texto para la leyenda.])
```

- `color`: el color de la barra, del borde de la nota, del número y del marco. El fondo de la nota se calcula solo (`color.lighten(91%)`) y el relleno del marco también (`transparentize(92%)`).
- `rotulo`: la palabra en mayúsculas que encabeza cada nota. Corta (≤ 11 letras).
- `desc`: una línea para la leyenda de la portada.

No edites `motor.typ`: extiende `tipos-base` en tu archivo y pásalo con `tipos:`.

```
#import "../motor.typ": *
// agregar tipos
#let mis-tipos = tipos-base + (
  definicion: (color: rgb("#0f766e"), rotulo: "DEFINICIÓN", desc: [Definiciones
clave.]),
  pregunta:   (color: rgb("#be185d"), rotulo: "PREGUNTA",   desc: [Preguntas para el
grupo.]),
)
// cambiar el color o el rótulo de un tipo existente
#mis-tipos.at("cuidado").insert("color", rgb("#c2410c"))
```

```
// quitar un tipo
#let _ = mis-tipos.remove("ejercicio")
// usarlos
#let pag = anotador("/x.pdf", W: 612, H: 792, xr: 547, tipos: mis-tipos)
#portada([Título], [Subtítulo], [Intro], W: 612, H: 792, tipos: mis-tipos)
```

Para un conjunto completamente distinto, define un diccionario nuevo desde cero con la misma forma. El orden del diccionario es el orden de la leyenda.

Consejos de color. Usa tonos medios u oscuros (luminosidad 30–45 %): con fondo al 91 % de blanco el texto sigue legible y la barra se distingue. Evita dos tipos del mismo matiz. No pases de 6–7 tipos: los colores dejan de ser memorables. Los de base son azul `#1f4e79`, verde `#2e6b30`, naranjo `#b5530f`, morado `#5b2a86` y gris `#5a5a5a`; buenos agregados: verde azulado `#0f766e`, magenta `#be185d`, ocre `#a16207`.

6. Márgenes, columnas y figuras

- **Una columna:** solo el margen derecho; `xr` = borde derecho del texto + 4.
- **Dos columnas:** las notas de la columna derecha van a la derecha (`xr` = borde de esa columna + 4). Para la columna izquierda usa `ML` de 140–180 pt, `xl` = borde izquierdo de la columna izquierda y `lado: "izq"`. Así ninguna conexión cruza el texto.
- **Páginas con figuras grandes o el texto más angosto:** cambia `xr` por página para que las barras no caigan encima de una figura. Para señalar la figura usa `r:` alrededor de ella o de su pie.
- El blanco que queda entre `xr` y el borde del original se usa para la ficha numerada; si `xr` queda a menos de 14 pt del borde de la página, la ficha invade el margen gris (no es grave).

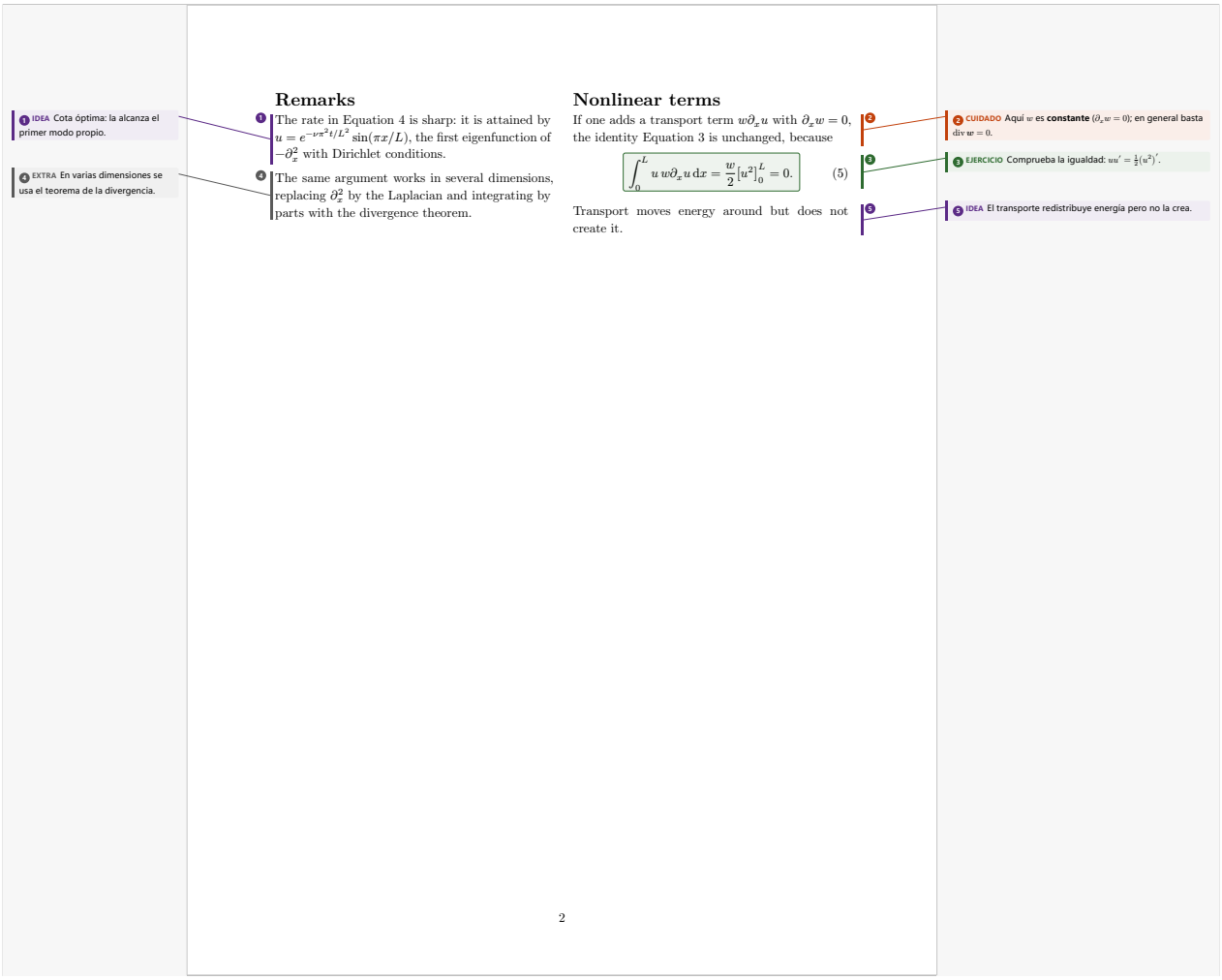


Figura 1: Página 2 del ejemplo (a dos columnas), anotada con ambos márgenes y un tipo con el color cambiado (CUIDADO).

7. Cómo escribir buenas notas

- **Una idea por nota**, en 1–3 líneas. El margen tiene unos 200 pt de ancho a 7.3 pt de letra (unos 55 caracteres por línea).
- **Densidad:** hasta unas 10–12 notas por página en un paper de una columna. Si hay `DESBORDE`, primero acorta notas, después fusiona notas vecinas y solo al final baja `tam` (no menos de 6.8 pt).
- **Usa el color por intención**, no por adorno: el lector debe poder saltar todas las notas grises, por ejemplo.
- **Remite, no repitas.** Si existe un material de apoyo (apunte, libro), la nota dice *dónde* está explicado («§4.4.2», «(RO)») y agrega solo lo que falta en el original. Pon en la portada una tabla con las etiquetas que usas (`derecha:`).
- **Señala trampas de notación** (convenciones traspuestas, promedios en vez de integrales, nombres de componentes cambiados): es lo que más le ahorra tiempo a un lector.
- Si el original tiene un error o una diferencia numérica, dilo con precisión y explica cómo lo verificaste.
- Prioriza el anclaje de ecuaciones numeradas y de los enunciados de teoremas; los párrafos se anclan por su primera y última línea.

8. Problemas frecuentes

Síntoma	Causa y solución
«entry path must be a valid virtual path» / «file not found»	Falta `--root`, o el PDF original está fuera de la raíz. Pon ambos bajo una misma raíz y usa una ruta absoluta desde ella.
«unknown variable: Div» (o cualquier palabra en `$...$`)	En la matemática de Typst las palabras de varias letras son variables. Usa `op("div")`, `"texto"` o define atajos (`#let Div = math.op("div")`).
«unknown symbol modifier» con `angle.l`	En Typst ≥ 0.13 se llama `chevron.l` / `chevron.r`. Tampoco existe `qquad`: usa `wide`.
Barras corridas respecto del texto	El PDF tiene `cropbox` ≠ `mediabox`. Prueba: dibuja `rect` en el bbox de una palabra que dé `get_text("words")` y compara. Si hay desfase constante, réstalo a todas las `y`.
Página sin capa de texto (0 palabras)	Es un escaneo. Usa `cuadricula` y lee las `y` a ojo, o aplica OCR antes (por ejemplo `ocrmypdf`, si está instalado y el usuario lo autoriza).
`DESBORDE`	Demasiadas notas o muy largas para el alto de la página: acorta, fusiona o divide la página en dos lados (`ML`).
Una conexión cruza una figura	Cambia `xr` en esa página para que las barras queden a la derecha de la figura.
Archivo de salida muy pesado	Los escaneos incrustados pesan lo mismo que el original; es normal.

9. Esqueleto mínimo para un documento nuevo

```
#import "/Plantilla de anotaciones/motor.typ": *   // o una ruta relativa
#set document(title: "Paper X anotado")

#let src = "/papers/paper-x.pdf"        // absoluta desde --root
#let W = 595.28                          // de `herramientas.py info`
#let H = 841.89
#let pag = anotador(src, W: W, H: H, xr: 520)

#portada([Paper X, anotado], [Autor, revista, año], [Para qué son estas notas.], W: W,
H: H)

#pag(1, (
  n(120, 180, "idea")[Lo que el resumen promete, en una frase.],
  n(400, 430, "apunte", r: (150, 400, 500, 430))[La ecuación (3) está explicada en
§2.1.],
))
#pag(2, (
  n(90, 140, "cuidado")[Aquí el autor usa la convención traspuesta.],
))
```

Lista de control antes de entregar: (1) compila sin errores; (2) `revisar` no reporta DESBORDE; (3) miraste cada página renderizada; (4) cada referencia «§…» existe en el material de apoyo; (5) la portada explica los colores; (6) el original quedó intacto, porque el motor nunca lo modifica.